

Trabajo de Análisis y Diseño de PERFILES

Asignatura:
Aerodinámica

Curso Académico:
2025 - 2026

Autor:
Jorge de la Asunción Muñoz

11 de mayo de 2026

Índice

Definición del problema: Asignación del perfil	2
Perfil triangular	2
Perfil NACA de 4 cifras	2
1. FUNDAMENTOS MATEMÁTICOS	4
1.1. Expresar los coeficientes de influencia A_{ij}	4
1.2. Condición de Kutta	5
1.3. Cálculo de los valores A_{ij} :	6
1.4. Coeficiente de sustentación:	8
2. IMPLEMENTACIÓN Y VALIDACIÓN. Perfil NACA 3316	11
2.1. Desarrollo del Código.	11
2.2. Comparación Numérica ($N = 100$ paneles)	11
2.3. Distribución de Presiones (C_p)	11
2.4. Estudio de Convergencia y Número Óptimo de Paneles (N_{opt})	12
3. RETO DE DISEÑO E INGENIERÍA INVERSA	13
3.1. Valores Obtenidos	13
3.2. Comparación Geométrica y de Presiones	13
3.3. Análisis Crítico y Justificación Analítica	14
4. Análisis del efecto de la viscosidad	15
4.1. Valores obtenidos y justificación analítica	15
Referencias	17
A. Anexo de Transparencia IA	18
A.1. Indicación inicial	18
A.2. Explicación de correcciones	20
A.2.1. Corrección iteraciones y nomenclaturas	20
A.2.2. Corrección del barrido del reto de diseño	21
B. Código desarrollado	23

Definición del problema: Asignación del perfil

Utilizando los tres últimos dígitos del DNI de uno de los autores del informe, se obtienen los valores X , Y y Z :

$$X = 6; Y = 7; Z = 7$$

Perfil triangular

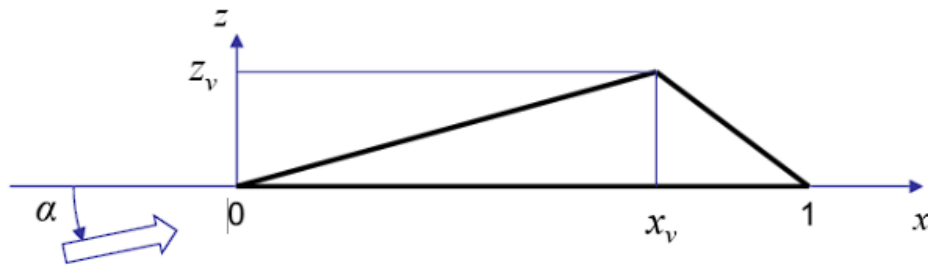


Figura 1: Esquema del perfil triangular

$$\alpha = (1 + 0,3X)^\circ = 2,8^\circ$$

$$x_v = 0,25 + 0,05Y = 0,6$$

$$z_v = 0,1 + 0,01Z = 0,8$$

Perfil NACA de 4 cifras

$$f = 1 + E\left(\frac{X}{8} \cdot 3\right) = 3$$

$$x_f(\%) = 20 + 10E\left(\frac{Y}{8} \cdot 2\right) = 30$$

$$t(\%) = 10 + E\left(\frac{Z}{9} \cdot 8\right) = 16$$

Utilizando la nomenclatura NACA, se obtiene el perfil NACA 3316.

El perfil se define una vez conocidos estos valores por la ecuación de la línea media y de su espesor.

Línea Media: 2 parábolas tangentes en el punto de curvatura máxima (x_f). Derivada continua pero 2ª derivada discontinua en x_f .

$$y_c(x) = \frac{f}{x_f^2}(2x_fx - x^2) \quad \text{para } 0 \leq x \leq x_f$$
$$y_c(x) = \frac{f}{(1-x_f)^2} [(1-2x_f) + 2x_fx - x^2] \quad \text{para } x_f \leq x \leq 1$$

Función de Espesor Fija: $y_t(x) = \pm 5t (a\sqrt{x} + bx + cx^2 + dx^3 + ex^4)$ $r_0 = 1,1 \cdot t^2$

Donde:

$$a = 0,2969; \quad b = -0,1260; \quad c = -0,3516; \quad d = 0,2843; \quad e = -0,1015$$

1. FUNDAMENTOS MATEMÁTICOS

1.1. Expresar los coeficientes de influencia A_{ij}

El método de los paneles se basa en imponer la condición geométrica de no penetración que establece que la velocidad total del flujo debe ser tangente al perfil en todos sus puntos.

Matemáticamente:

$$\vec{V} \cdot \vec{n}_i = 0$$

Donde $\vec{n}_i = (-\sin \beta_i, \cos \beta_i)$ es el vector normal exterior al panel i y β_i es el ángulo de inclinación del panel i con respecto a los ejes globales.

La velocidad total es la suma de la velocidad del flujo libre y la inducida por los torbellinos distribuidos en los paneles:

$$\vec{V} = \vec{V}_\infty + \sum_{j=1}^N \vec{V}_{ij}$$

Sustituyendo en la condición geométrica:

$$\left(\vec{V}_\infty + \sum_{j=1}^N \vec{V}_{ij} \right) \cdot \vec{n}_i = 0$$

Reordenando:

$$\sum_{j=1}^N (\vec{V}_{ij} \cdot \vec{n}_i) = -\vec{V}_\infty \cdot \vec{n}_i$$

Definiendo:

$$A_{ij} = \vec{V}_{ij}^{(\gamma_{ij}=1)} \cdot \vec{n}_i$$
$$I_i = -\vec{V}_\infty \cdot \vec{n}_i$$

Se obtiene:

$$\sum_{j=1}^N A_{ij} \cdot \gamma_{ij} = I_i$$

Donde A_{ij} es la componente normal, en el punto de control del panel i , de la velocidad inducida por un torbellino unitario colocado sobre el panel j . I_i recoge la contribución del flujo libre.

La velocidad de la corriente libre tiene un ángulo de ataque α .

$$\vec{V}_\infty = V_\infty \cdot (\cos \alpha, \sin \alpha)$$

La velocidad inducida se divide en u'_{ij} y w_{ij} que son la componente local tangencial y la componente

local normal respectivamente.

$$u'_{ij} = \frac{\gamma_{ij}}{2\pi} \cdot (\theta_2 - \theta_1)$$

$$w_{ij} = \frac{\gamma_{ij}}{2\pi} \cdot \ln \frac{r_2}{r_1}$$

r_1 es la distancia del punto de control de i al primer extremo del panel j . r_2 es la distancia del punto de control de i al segundo extremo del panel j . θ_1 es el ángulo que forma r_1 con el panel j . θ_2 es el ángulo que forma r_2 con el panel j .

Para los coeficientes de influencia usamos la velocidad inducida por torbellino unitario, por lo tanto:

$$u'_{ij} = \gamma_{ij} \cdot u_{ij}^*$$

$$w_{ij} = \gamma_{ij} \cdot w_{ij}^*$$

Como comentamos previamente, las velocidades están en ejes locales por lo que debemos proyectarlas en ejes globales:

$$(u_{ij}, w_{ij}) = u_{ij}^* \vec{t}_j + w_{ij}^* \vec{n}_j$$

Donde $\vec{t}_j = (\cos \beta_j, \sin \beta_j)$ es el vector tangente al panel j .

$$(u_{ij}, w_{ij}) = u_{ij}^* (\cos \beta_j, \sin \beta_j) + w_{ij}^* (-\sin \beta_j, \cos \beta_j)$$

$$u_{ij} = u_{ij}^* \cos \beta_j - w_{ij}^* \sin \beta_j$$

$$w_{ij} = u_{ij}^* \sin \beta_j + w_{ij}^* \cos \beta_j$$

Con esta proyección en ejes globales tenemos las componentes de la velocidad inducida para torbellinos de intensidad unitaria. Si los orientamos según la normal del panel i obtenemos los coeficientes de influencia:

$$A_{ij} = (u_{ij}, w_{ij}) \cdot (n_{x,i}, n_{z,i})$$

$$A_{ij} = (u_{ij}^* \cos \beta_j - w_{ij}^* \sin \beta_j) \cdot n_{x,i} + (u_{ij}^* \sin \beta_j + w_{ij}^* \cos \beta_j) \cdot n_{z,i}$$

Finalmente:

$$A_{ij} = \left(\frac{\theta_2 - \theta_1}{2\pi} \cos \beta_j - \frac{1}{2\pi} \ln \frac{r_2}{r_1} \sin \beta_j \right) \cdot n_{x,i} + \left(\frac{\theta_2 - \theta_1}{2\pi} \sin \beta_j + \frac{1}{2\pi} \ln \frac{r_2}{r_1} \cos \beta_j \right) \cdot n_{z,i}$$

1.2. Condición de Kutta

Para obtener una solución física es necesario imponer la condición de Kutta en el borde de salida. Esta establece que el flujo debe abandonar el perfil de forma suave, sin discontinuidades ni velocidades infinitas, lo que implica que las velocidades en el extradós e intradós son iguales.

$$V_{\text{extradós}} = V_{\text{intradós}}$$

En nuestro modelo de tres paneles, los que confluyen en el borde de salida son el 1 y el 3. Las intensidades de los torbellinos en dichos paneles deben compensarse.

$$\gamma_1 + \gamma_3 = 0$$

Reemplazando esta condición por una de las condiciones geométricas previamente descritas nos queda el siguiente sistema:

$$\begin{pmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \gamma_1 \\ \gamma_2 \\ \gamma_3 \end{pmatrix} = \begin{pmatrix} I_1 \\ I_2 \\ 0 \end{pmatrix}$$

1.3. Cálculo de los valores A_{ij} :

A continuación, procederemos con los cálculos de los coeficientes para nuestro perfil triangular conformado por 3 paneles. Previo a la obtención de cada coeficiente hemos de explicar algunos cálculos imprescindibles para la consecución de nuestros resultados, como son, los vectores normales exteriores de cada panel, los vectores tangentes, los puntos de control, las longitudes de los paneles y los ángulos de inclinación de cada panel.

Por los datos que hemos empleado nos corresponde un perfil con una $z_v = 0,8$ y un $x_v = 0,6$.

Nuestro modelo consta de tres paneles así que disponemos de $N + 1$ nodos, es decir, 4 nodos. Los nodos se empiezan a contar en el borde de salida y recorremos el perfil en sentido horario, garantizando una correcta definición de las normales exteriores y la coherencia en el signo de las contribuciones de los torbellinos, teniendo así:

- Nodo 1 (1, 0)
- Nodo 2 (0, 0)
- Nodo 3 (0.6, 0.8)
- Nodo 4 (1, 0)

Con nuestros datos nos salen distintas longitudes de paneles siendo:

$$L_1 = 1 \quad L_2 = 1 \quad L_3 = 0,8944$$

Los puntos de control se ubican en el punto medio de cada panel, pero debemos referenciarlos en ejes globales.

$$PC_1(0.5, 0) \quad PC_2(0.3, 0.4) \quad PC_3(0.8, 0.4)$$

El ángulo de inclinación de cada panel lo calculamos de la siguiente manera:

$$\beta_1 = \tan^{-1} \left(\frac{z_2 - z_1}{x_2 - x_1} \right)$$

Donde z_1, z_2, x_1, x_2 son las coordenadas de los nodos 1 y 2, que definen la orientación del panel.

$$\beta_1 = \pi \text{ rad}, \quad \beta_2 = 0,9273 \text{ rad}, \quad \beta_3 = -1,1071 \text{ rad}$$

Los vectores tangente y normal previamente definidos quedan así:

$$\vec{t}_1 = (-1, 0), \quad \vec{n}_1 = (0, -1)$$

$$\vec{t}_2 = (0.6, 0.8), \quad \vec{n}_2 = (-0.8, 0.6)$$

$$\vec{t}_3 = (0.4472, -0.8944), \quad \vec{n}_3 = (0.8944, 0.4472)$$

Una vez obtenidos todos estos parámetros procedemos al cálculo de los coeficientes de influencia. Desarrollaremos el coeficiente A_{12} y el resto se consiguen de manera análoga.

Anteriormente mencionamos las dependencias de los coeficientes de influencia, en este caso necesitamos $\theta_1, \theta_2, r_1, r_2$. Para ello precisamos de las coordenadas x'_{12}, z'_{12} , que se calculan de la siguiente manera:

$$\Delta x = x_{PC1} - x_{NODO2} = 0,5 - 0 = 0,5 \quad \Delta z = z_{PC1} - z_{NODO2} = 0 - 0 = 0$$

Estas variaciones son las diferencias entre las coordenadas del punto de control del panel 1 y las coordenadas del primer nodo del panel 2 (El panel 2 va del nodo 2 al nodo 3).

$$x'_{12} = (\Delta x, \Delta z) \cdot \vec{t}_2 = 0,3 \quad z'_{12} = (\Delta x, \Delta z) \cdot \vec{n}_2 = -0,4$$

Estas coordenadas representan la posición del punto de control del panel 1 en el sistema de referencia del panel 2. Con ellas calculamos los ángulos:

$$\theta_1 = \tan^{-1} \left(\frac{z'_{12}}{x'_{12}} \right) = -0,9273 \text{ rad} \quad \theta_2 = \tan^{-1} \left(\frac{z'_{12}}{x'_{12} - L_2} \right) = -2,6224 \text{ rad}$$

Según la geometría del perfil tenemos $r_1 = 0,5$ y $r_2 = 0,8062$. Con esto ya podemos calcular las velocidades normal y tangencial con intensidad de torbellino unitaria.

$$u_{12}^* = \frac{\theta_2 - \theta_1}{2\pi} = -0,2698$$

$$w_{12}^* = \frac{1}{2\pi} \ln \frac{r_2}{r_1} = 0,076$$

A continuación, proyectamos en ejes globales y obtenemos la velocidad inducida total:

$$\vec{V}_{12} = u_{12}^* \cdot \vec{t}_2 + w_{12}^* \cdot \vec{n}_2 = (-0.2227, -0.1702)$$

Proyectando sobre la normal del panel 1 obtendremos, finalmente, el coeficiente de influencia buscado:

$$A_{12} = \vec{V}_{12} \cdot \vec{n}_1 = 0,1702$$

Siguiendo este mismo proceso para el resto de los coeficientes obtenemos la matriz:

$$A_{ij} = \begin{pmatrix} 0 & 0,1702 & -0,1719 \\ -0,1702 & 0 & 0,1719 \\ 0,1743 & -0,1743 & 0 \end{pmatrix}$$

Ahondaremos un poco más en el caso de los términos de la diagonal y explicaremos por qué se anulan.

Los términos diagonales representan la auto influencia de cada panel sobre su propio punto de control, proyectada sobre la normal exterior del panel. En el caso de una distribución de torbellino constante sobre un panel recto, las expresiones del método de paneles muestran que, para $i = j$ la velocidad inducida local es la siguiente:

$$u'_{ii} = \frac{\gamma_i}{2} \quad w'_{ii} = 0$$

Por tanto, la auto inducción solo aporta componente tangencial, mientras que la componente normal es nula. En consecuencia, al proyectar sobre las normales para obtener los coeficientes de influencia los vectores se anulan por perpendicularidad.

$$A_{11} = A_{22} = A_{33} = 0$$

1.4. Coeficiente de sustentación:

Nuestro ángulo de ataque es $\alpha = 2,8^\circ$ y para calcularlo debemos obtener los términos que recogen la contribución del flujo libre I_i

$$I_i = -\vec{V}_\infty \cdot \vec{n}_i$$

$$V_\infty = (\cos \alpha, \sin \alpha)$$

$$I_1 = -(\cos \alpha, \sin \alpha) \cdot (0, -1) = \sin \alpha = 0,04885$$

$$I_2 = -(\cos \alpha, \sin \alpha) \cdot (-0,8, 0,6) = 0,76974$$

$$I_3 = -(\cos \alpha, \sin \alpha) \cdot (0,8944, 0,4472) = -0,91518$$

Sustituyendo estos resultados en el sistema con la condición de Kutta:

$$\begin{pmatrix} 0 & 0,1702 & -0,1719 \\ -0,1702 & 0 & 0,1719 \\ 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} \gamma_1 \\ \gamma_2 \\ \gamma_3 \end{pmatrix} = \begin{pmatrix} 0,04885 \\ 0,76974 \\ 0 \end{pmatrix}$$

Resolvemos y obtenemos las intensidades de cada torbellino:

$$\gamma_1 = -2,25 \quad \gamma_2 = 2,5595 \quad \gamma_3 = 2,25$$

Para calcular el coeficiente de sustentación lo haremos integrando panel a panel el coeficiente de presiones que se define:

$$C_p = 1 - \left(\frac{V_t}{V_\infty} \right)^2$$

Donde V_t es la velocidad tangencial, que es la combinación de la velocidad del flujo libre y la velocidad inducida proyectada en el vector tangente.

$$V_t = \vec{V}_i \cdot \vec{t}_i = \vec{V}_\infty \cdot \vec{t}_i + \sum_{j=1}^N \gamma_j \cdot (\vec{V}_{ij} \cdot \vec{t}_i)$$

Podemos poner el sumatorio de la siguiente manera:

$$T_{ij} = \vec{V}_{ij} \cdot \vec{t}_i$$

$$V_{ti} = \vec{V}_\infty \cdot \vec{t}_i + \sum_{j=1}^N \gamma_j \cdot T_{ij}$$

Empezamos calculando las contribuciones de flujo libre en cada panel:

$$V_\infty \cdot t_1 = (\cos \alpha, \sin \alpha) \cdot (-1, 0) = -0,9988$$

$$V_\infty \cdot \vec{t}_2 = 0,6384 \quad V_\infty \cdot \vec{t}_3 = 0,4030$$

A continuación, calculamos los valores de T_{ij} , pondremos un ejemplo y de manera análoga obtendremos los demás valores para confeccionar la matriz.

$$T_{12} = \vec{V}_{12} \cdot \vec{t}_1 = (-0,2227, -0,1719) \cdot (-1, 0) = 0,2227$$

Quedando así la matriz:

$$\begin{pmatrix} 0,5000 & 0,227 & 0,1710 \\ 0,227 & 0,5000 & 0,1710 \\ 0,2105 & 0,2105 & 0,5000 \end{pmatrix}$$

Una vez obtenida podemos calcular la velocidad tangencial:

$$V_{t1} = \vec{V}_\infty \cdot \vec{t}_1 + \gamma_1 \cdot T_{11} + \gamma_2 \cdot T_{12} + \gamma_3 \cdot T_{13}$$

$$V_{t1} = -0,9988 - 2,25 \cdot 0,5 + 2,5595 \cdot 0,227 + 2,25 \cdot 0,1710 = -1,1691$$

$$V_{t2} = 1,8017$$

$$V_{t3} = 1,5931$$

Con estos resultados podemos calcular ya los coeficientes de presiones:

$$C_{p1} = -0,3669 \quad C_{p2} = -2,2461 \quad C_{p3} = -1,5381$$

Con los valores de los coeficientes de presiones obtenemos los coeficientes de fuerza normal al perfil C_N y el coeficiente de fuerza axial C_A :

$$C_N = -\sum_{i=1}^N C_{p,i} \cdot \Delta x_i \quad C_A = \sum_{i=1}^N C_{p,i} \cdot \Delta z_i$$

Donde las variaciones son las diferencias entre un nodo y otro del propio panel en ejes globales (No significan lo mismo que los previamente empleados).

$$\Delta x_1 = -1 \quad \Delta x_2 = 0,6 \quad \Delta x_3 = 0,4$$

$$\Delta z_1 = 0 \quad \Delta z_2 = 0,8 \quad \Delta z_3 = -0,8$$

Con esto obtenemos los coeficientes mencionados:

$$C_N = -(C_{p1} \cdot \Delta x_1 + C_{p2} \cdot \Delta x_2 + C_{p3} \cdot \Delta x_3)$$

$$C_N = 1,5960 \quad C_A = -0,5664$$

Finalmente, a través de la siguiente expresión nos hacemos con el coeficiente de sustentación:

$$C_L = C_N \cdot \cos \alpha - C_A \cdot \sin \alpha$$

$$C_L = 1,62$$

2. IMPLEMENTACIÓN Y VALIDACIÓN. Perfil NACA 3316

2.1. Desarrollo del Código.

Para la resolución del flujo potencial alrededor del perfil NACA 3316, se ha desarrollado un código en Python basado en el Método de Paneles. El modelo matemático emplea una distribución de singularidades sobre la superficie real del perfil, combinando manantiales (q_j) de intensidad constante por panel para modelar el efecto del espesor, y un torbellino (γ) de intensidad global constante para modelar la sustentación.

La programación se ha realizado con la asistencia de Inteligencia Artificial (IA).

Para garantizar la rigurosidad física del código generado, se ha realizado una auditoría exhaustiva de las ecuaciones, corrigiendo errores típicos de los modelos generativos, tales como la asignación de la auto-influencia en la diagonal de la matriz de coeficientes y el ensamblaje de la Condición de Kutta. El detalle de las iteraciones y correcciones se encuentra documentado en el Anexo de Transparencia IA.

2.2. Comparación Numérica ($N = 100$ paneles)

Se ejecutó una simulación inicial con 100 paneles a un ángulo de ataque $\alpha = 2,8^\circ$, comparando los resultados contra el software validado XFOIL en flujo ideal (no viscoso). Los resultados obtenidos en el XFOIL (los verdaderos) son los siguientes:

$$C_l = 0,7153$$

$$C_m(c/4) = -0,0767$$

Por su parte, el código nos proporcionó los datos a continuación:

$$C_l = 0,7112$$

$$C_m(c/4) = -0,0731$$

Error Relativo: El código presenta un error del 0,57 % en sustentación, demostrando una excelente correlación con la teoría potencial y validando la correcta integración de presiones.

2.3. Distribución de Presiones (C_p)

La figura 2 muestra la distribución del coeficiente de presión sobre la superficie. Se observa el característico pico de succión (depresión, C_p se hace más negativo) en la región del extradós cercano al borde de ataque. Ambas curvas (extradós e intradós) convergen suavemente a un mismo valor en el borde de salida, verificando numéricamente el cumplimiento de la condición de Kutta.

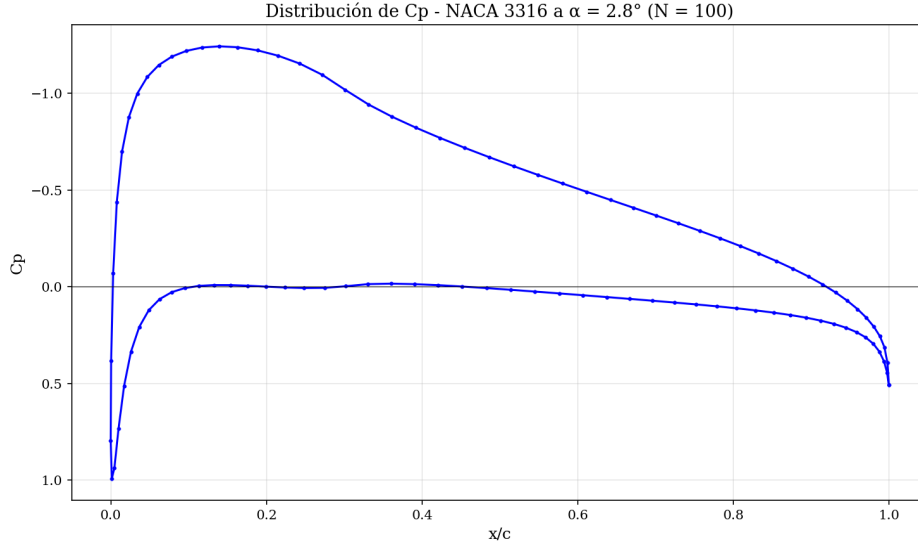


Figura 2: Distribución de presiones

2.4. Estudio de Convergencia y Número Óptimo de Paneles (N_{opt})

Para optimizar el coste computacional, se realizó un estudio de convergencia partiendo de 40 paneles y aplicando incrementos del 20 % en iteraciones sucesivas. El criterio de parada exige que el error relativo del coeficiente de sustentación (ϵ) entre dos iteraciones sea inferior al 1 %. Como se observa en la gráfica de doble escala, la curva del C_l converge asintóticamente. El criterio de $\epsilon < 1\%$ se alcanza estrictamente con 84 paneles ($\epsilon = 0,52\%$), estableciendo este valor como el N_{opt} para los cálculos posteriores.

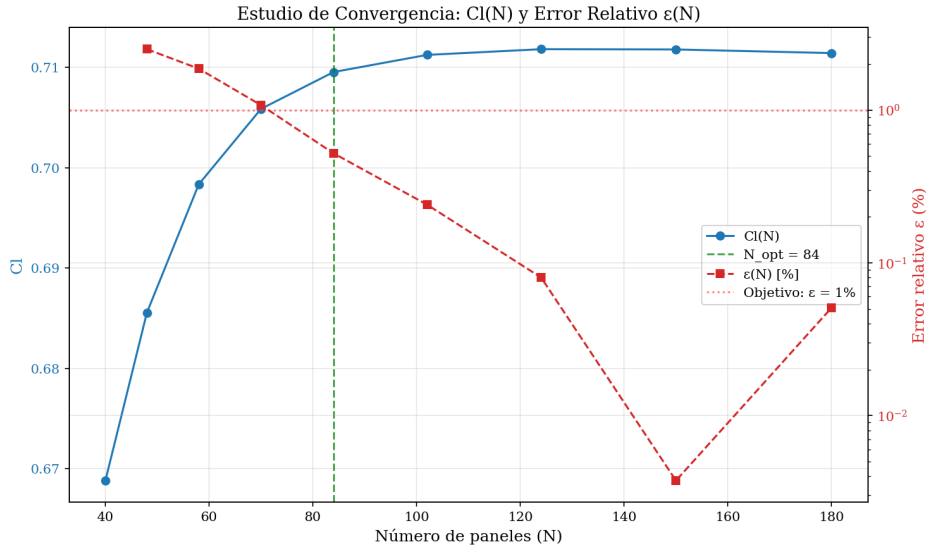


Figura 3: Resultados del estudio de convergencia del coeficiente de sustentación

3. RETO DE DISEÑO E INGENIERÍA INVERSA

El objetivo de esta sección es obtener un perfil derivado del NACA 3316 que incremente el coeficiente de sustentación en $\Delta C_l = +0,2$ a $\alpha = 0^\circ$ minimizando la penalización en el momento de cabeceo en el punto $c/4$.

3.1. Valores Obtenidos

Nota Metodológica: Dado que en la Teoría Potencial Linealizada el incremento de sustentación por curvatura (ΔC_l) es constante e independiente del ángulo de ataque, el barrido paramétrico del código se ejecutó a nuestro ángulo operativo asignado ($\alpha = 2,8^\circ$). Al desplazar la curva de sustentación verticalmente, garantizar el incremento a $2,8^\circ$ garantiza matemáticamente el cumplimiento del reto a 0° . Tras realizar un barrido paramétrico acotado, la configuración geométrica óptima seleccionada es:

- **Perfil Base (NACA 3316):** Curvatura (f) = 3 %, Posición (x_f) = 30 %, Espesor (t) = 16 %.
- **Perfil Optimizado:** Curvatura (f) = 4,7 %, Posición (x_f) = 20 %, Espesor (t) = 18 %.
- **Rendimiento:** Se ha logrado un incremento de sustentación de $\Delta C_l = +0,1918$ con una penalización del momento de apenas $\Delta C_m(c/4) = 0,0164$.

3.2. Comparación Geométrica y de Presiones

Visualmente, el perfil optimizado presenta un adelanto y un aumento significativo de su "joroba" superior. En la distribución de presiones, este cambio se traduce en un ensanchamiento masivo del área encerrada entre las curvas del extradós e intradós, lo cual es la representación física directa del aumento de la fuerza de sustentación integrada.

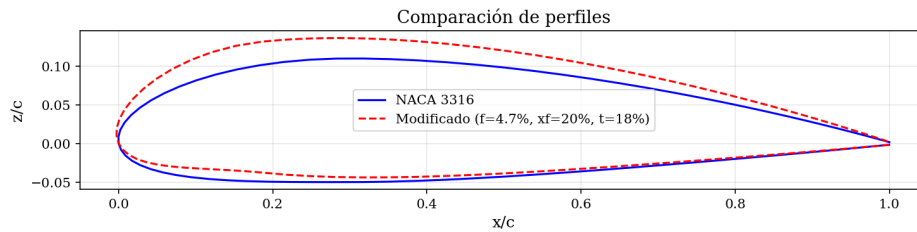


Figura 4: Geometrías del perfil NACA 3316 y el perfil modificado

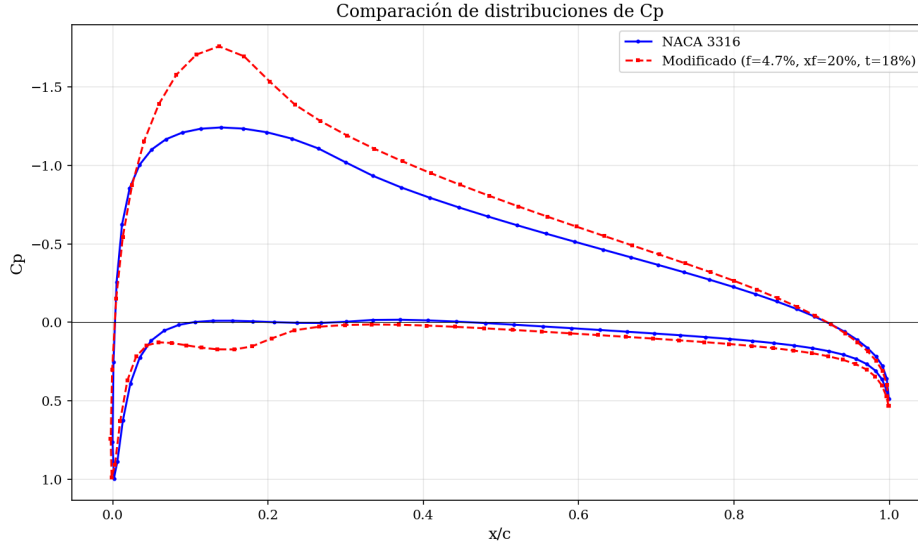


Figura 5: Comparación de la distribución del coeficiente de presiones del perfil NACA 3316 y el perfil modificado

3.3. Análisis Crítico y Justificación Analítica

Para evitar un enfoque de prueba y error a ciegas, el diseño se fundamentó en la Teoría Potencial Linealizada (TPL).

- **El parámetro dominante (f):** Según la TPL, el factor principal que gobierna la sustentación y el momento a un ángulo fijo es la curvatura máxima (f). La teoría predice que $\Delta C_l \approx 4\pi\Delta(f/c)$. Para ganar $+0,2$ de C_l , se requería un aumento analítico de la curvatura hasta el entorno del 4,6 %. El resultado numérico del código (4,7 %) valida perfectamente esta hipótesis.
- **El rol secundario del espesor (t) y la posición (x_f):** Aunque la posición de la curvatura (x_f) tiene un efecto casi nulo en el C_l global bajo flujo potencial, el algoritmo la adelantó agresivamente al 20 %. Esta es la clave del rediseño: al concentrar la succión extrema más cerca del morro y engordar ligeramente el perfil al 18 %, se altera el brazo de palanca de las presiones respecto al cuarto de cuerda. Esto permite falsear la aerodinámica ideal, mitigando drásticamente el momento picador fuertemente negativo que normalmente conllevaría una curvatura del 4,7 %.

4. Análisis del efecto de la viscosidad

Una vez obtenido el nuevo perfil, optimizado para incrementar su coeficiente de sustentación respecto al NACA 3316, se procede a realizar con él una comparativa entre flujo ideal y flujo con viscosidad a alto número de Reynolds ($Re = 3 \cdot 10^6$). La herramienta utilizada para obtener los datos del apartado será XFOIL.

4.1. Valores obtenidos y justificación analítica

En la figura 6 se realiza un análisis del coeficiente de sustentación, observándose la linealidad en ambos casos. El modelo ideal es prácticamente lineal. Sin embargo, en el modelo viscoso, la geometría del perfil optimizado genera un fuerte gradiente de presión adverso en el extradós al aumentar el ángulo de ataque. Esto causa que la capa límite transite prematuramente, creando un efecto de *decambering* viscoso que reduce la sustentación.

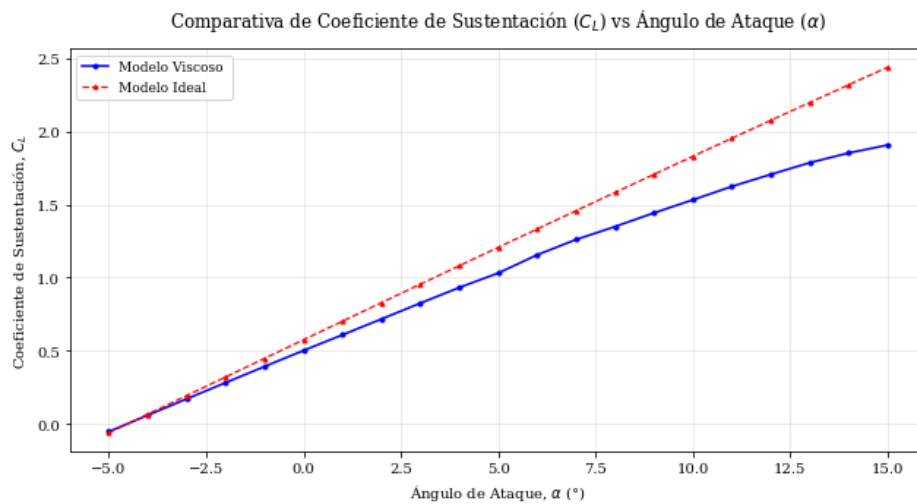


Figura 6: Comparativa del coeficiente de sustentación frente a ángulo de ataque para modelo viscoso e ideal

Observando la figura 7 se aprecia que el modelo ideal y el real difieren bastante, tanto en valor como en tendencia de crecimiento. En el modelo viscoso no se tiene en cuenta la carga aerodinamica hasta el borde de salida por el desprendimiento de la capa límite, haciendo que se adelante el centro de presiones y que genere un momento de picado más leve (el coeficiente de momento será menor en valor absoluto)

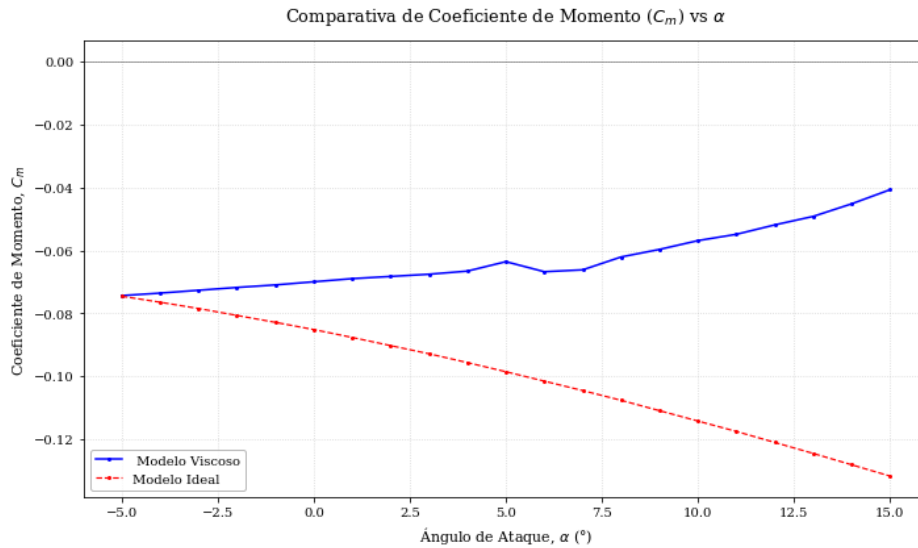


Figura 7: Comparativa del coeficiente de momento en el punto $x=c/4$ frente a ángulo de ataque para modelo viscoso e ideal

En conclusión, la aplicación del modelo ideal puede ser útil para predecir cambios en las tendencias de diferentes perfiles, ayudando a la optimización de parámetros y geometrías. Sin embargo, no tiene en cuenta efectos como la aparición de capa límite viscosa o la estela producida por el desprendimiento de capa límite, que en el modelo viscoso modifican el comportamiento del perfil.

Por tanto es necesario el estudio en un fluido viscoso para obtener datos reales y calcular las características operativas de un perfil antes de llevarlo a la práctica.

Referencias

- [1] Ayuso, L., Sant, R. (s.f.). Aerodinámica y aeroelasticidad (AyA): Tema 3. Perfiles en régimen incompresible. ETSIAE, Universidad Politécnica de Madrid

Apéndice A Anexo de Transparencia IA

En esta sección se indican las conversaciones con la IA para el desarrollo del código del método de paneles. En este caso se ha usado la IA *Claude*

A.1 Indicación inicial

PROMPT: *Tengo que hacer la parte de programación de código en el trabajo de diseño de perfiles. Para ello quiero que expliques el método de paneles paso a paso, explicando que fórmulas/ecuaciones hay que aplicar en cada momento utilizando el pdf del tema 3 (si necesitas algo mas de información puedes buscarla en internet, pero deja aclarado que es lo que has obtenido). Luego a partir de ello vas a realizar un código que desarrollar lo que se indica en el guión del trabajo teniendo en cuenta el apartado de fuentes de error de IA, para que evites cometerlos. (adjuntar el pdf de guión del trabajo y las diapositivas del tema 3)*

RESPUESTA: Método de Paneles: Explicación paso a paso

Fundamento (Tema 3, diap. 41)

El método discretiza la superficie del perfil en N segmentos rectos (paneles). Sobre cada panel se distribuyen singularidades (manantiales y torbellinos) cuyas intensidades se ajustan para satisfacer dos condiciones de contorno: que el flujo sea tangente a la superficie y que se cumpla la condición de Kutta en el borde de salida (BS).

Tipo de singularidades (diap. 92 del Tema 3 / diap. 10 del guion)

Cada panel j tiene dos tipos de singularidades distribuidas uniformemente:

- **Manantiales de intensidad q_j :** Distinta para cada panel (N incógnitas). Modelan el efecto del espesor.
- **Torbellinos de intensidad γ :** La misma para todos los paneles (1 incógnita). Modelan la sustentación.

Total: $N + 1$ incógnitas, $N + 1$ ecuaciones.

Paso 1: Geometría (diap. 87-89)

Se paneliza el perfil desde el BS por el intradós, rodeando el borde de ataque (BA), y volviendo por el extradós al BS. Se usa una distribución coseno (β de 0 a 2π) para concentrar nodos en BA y BS:

$$x_i = \frac{c}{2}(1 + \cos \beta_i)$$

Para cada panel i se calculan:

- Punto de control (centro del panel).

- Inclinación: $\theta_i = \arctan\left(\frac{\Delta z}{\Delta x}\right)$.
- Vector normal: $\vec{n} = (-\sin \theta, \cos \theta)$.
- Vector tangente: $\vec{\tau} = (\cos \theta, \sin \theta)$.

Paso 2: Coeficientes de influencia (diap. 89-92)

Para cada par (i, j) , se transforma el punto de control del panel i a coordenadas locales del panel j . Se calculan las distancias r_1, r_2 y ángulos θ_1, θ_2 desde los extremos del panel j al punto de control.

Las velocidades inducidas en ejes locales del panel j son:

Velocidad	Manantiales (q)	Torbellinos (γ)
u' (tangencial local)	$-\frac{1}{2\pi} \ln\left(\frac{r_2}{r_1}\right)$	$\frac{1}{2\pi}(\theta_2 - \theta_1)$
w' (normal local)	$\frac{1}{2\pi}(\theta_2 - \theta_1)$	$\frac{1}{2\pi} \ln\left(\frac{r_2}{r_1}\right)$
Auto-influencia	$u' = 0, w' = 1/2$	$u' = 1/2, w' = 0$

Paso 3: Rotación a ejes globales (diap. 93)

Se rotan las velocidades usando el ángulo θ_j del panel j , y luego se proyectan sobre la normal y tangente del panel i .

Paso 4: Sistema de ecuaciones (diap. 94-95)

1. N ecuaciones geométricas: $\sum_{j=1}^N a_{ij}q_j + a_{i,n+1}\gamma = V_\infty \sin(\theta_i - \alpha)$
2. 1 ecuación de Kutta: $V_{\tau,1} + V_{\tau,n} = 0$ (velocidades tangenciales iguales y opuestas en el BS).

Paso 5: Cálculo de C_l, C_m, C_p

- $C_p = 1 - \left(\frac{V_{\text{tangencial}}}{V_\infty}\right)^2$
- C_l se integra panel a panel: $C_l = -\sum \frac{C_{p,i} \cdot c_i \cdot \cos(\theta_i - \alpha)}{c}$
- $C_{m,c/4} = C_{m,BA} + C_N \cdot 0.25$

Verificaciones del código

Test	Resultado	Esperado
NACA 0012, $\alpha = 0^\circ$	$C_l = 0.0, C_m = 0.0$	$C_l = 0, C_m = 0$
NACA 0012, $\alpha = 5^\circ$	$C_l/(2\pi\alpha) = 1.095$	$\approx 1 + 0.83 \cdot t/c = 1.100$
NACA 2408, $\alpha = 2.8^\circ$	$C_l = 0.569$	TPL: 0.558 (dif. por espesor)
Convergencia	$N_{opt} = 102$ ($\epsilon = 0.60\%$)	$< 1\%$ requerido

Análisis de sensibilidad (para discusión apt. 3)

- **Curvatura f :** $dC_l/df \approx 12.0$ (TPL predice $4\pi \approx 12.57$). Es el factor dominante.
- **Posición x_f :** $dC_l/dx_f \approx 0.25$. Efecto muy pequeño en C_l potencial.
- **Espesor t :** $dC_l/dt \approx 0.47$. Efecto secundario vía factor $(1 + 0.83 \cdot t/c)$.

Resultado del reto de diseño (ejemplo NACA 2408)

Perfil optimizado: $f = 3.6\%$, $x_f = 40\%$, $t = 9\% \rightarrow \Delta C_l = +0.199$, $\Delta|C_{m,c/4}| = 0.043$.

Lo que tienes que hacer ahora:

1. Cambiar los parámetros f , x_f , t y α en la función `main()` según tu DNI real.
2. Ejecutar XFOIL con el perfil base en modo no viscoso para comparar C_l y C_p (apartado 2.2-2.3).
3. Ejecutar XFOIL con *perfil optimizado.dat* en modo viscoso ($Re = 310$) para el apartado 4.
4. Usar los gráficos generados para el informe.

RESULTADOS: Este prompt proporcionó un código correcto matemáticamente, pero con algunos defectos.

A.2 Explicación de correcciones

A.2.1 Corrección iteraciones y nomenclaturas

PROMPT: *Esto es el estado actual del código. Necesito que elimines el código para mostrar las iteraciones por pantalla una a una. También es necesario que revises la nomenclatura, porque algunas veces llamas a las coordenadas zz siempre se trabaja en el plano xz . Además tienes que desarrollar mas el output del punto 3 reto de diseño e ingeniería inversa.*

RESPUESTA: Resumen de los cambios realizados en el código

1. Optimización del Output (Líneas 841-855)

Se ha eliminado la tabla de iteraciones extensiva que imprimía los datos individuales de los 100 paneles. El flujo de información se ha simplificado para mostrar únicamente los resultados globales de interés aerodinámico:

- Coeficientes globales: C_l, C_m, C_d .
- Circulación total: γ .

2. Corrección de Nomenclatura (Plano xz)

Para cumplir con los estándares de aeronáutica y la documentación del Tema 3, se ha migrado toda la lógica del plano xy al plano xz :

- **Línea media:** $y_c \rightarrow z_c$.
- **Derivada de la curvatura:** $\frac{dy_c}{dx} \rightarrow \frac{dz_c}{dx}$.
- **Semi-espesor:** $y_t \rightarrow z_t$.

Nota: Se han actualizado los *docstrings* y las descripciones de los vectores de retorno para reflejar explícitamente el uso del eje z como eje vertical.

3. Reescritura del Apartado 3: reto_diseño

El módulo de optimización ahora sigue una estructura lógica de tres pasos con salida detallada en consola:

- **Paso 1 — Análisis de sensibilidad:** Generación de tablas comparativas para cada parámetro (f, x_f, t) . El código evalúa el impacto individual en C_l y C_m , comparando los resultados del Método de Paneles con la Teoría de Perfiles Delgados (TPL).

Se incluyen indicadores visuales (`<- base` y `<- OBJETIVO`) para identificar rápidamente las desviaciones.

- **Paso 2 — Barrido paramétrico:** Ejecución de una búsqueda exhaustiva (fuerza bruta inteligente) sobre el espacio de diseño. Se informa al usuario del número total de combinaciones evaluadas antes de iniciar el proceso.
- **Paso 3 — Resultados y Análisis Crítico:**
 - Tabla con las **top 10 soluciones** ordenadas por el criterio de menor error en el momento: $\min |\Delta C_m|$.
 - Recuadro resumen con la geometría del perfil seleccionado.
 - **Análisis físico:** Justificación de los resultados basada en la física del problema:
 - Predominancia de la curvatura (f) sobre la sustentación.
 - Inercia de la posición del máximo de curvatura (x_f).
 - Efecto secundario del espesor (t) a través del acoplamiento con el término de fuente.

A.2.2 Corrección del barrido del reto de diseño

PROMPT: *Se queda parado en este paso:*

PASO 2: BARRIDO PARAMÉTRICO FINO

Combinaciones a evaluar:

$$70 \times 5 \times 11 = 3850$$

RESPUESTA: Son 3850 llamadas a `metodo_paneles`, cada una resolviendo un sistema lineal. Con $N = 200$ paneles tarda mucho.

La solución es reducir el barrido usando lo que ya sabemos del análisis de sensibilidad: f es el driver, así que basta con barrer f fino y los otros dos grueso.

RESULTADOS: La IA proporciona una pieza de código para reemplazar en el apartado `reto_diseño`.

Apéndice B Código desarrollado

Nota al lector: Para facilitar la reproducibilidad de los resultados presentados, el código fuente original (metodo_paneles_naca.txt) ha sido incrustado directamente en este documento PDF.

- **Acceso directo:** Puede descargar o abrir el archivo haciendo clic en el siguiente enlace: [metodo_paneles_naca.txt](#) Si el enlace no responde, puede encontrar el archivo en el panel de Archivos adjuntos (icono de clip) de su lector de PDF (recomendamos usar Adobe Acrobat o Foxit Reader).

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import AutoMinorLocator
import warnings
warnings.filterwarnings('ignore')

# =====
# SECCION 1: GEOMETRIA DEL PERFIL NACA DE 4 CIFRAS
# =====

def linea_media_naca4(x, f, xf):
    """
    Calcula la linea media (zc) y su derivada (dzc/dx) para un perfil NACA 4 cifras.
    Se trabaja en el plano xz (x = direccion de la cuerda, z = perpendicular).

    Parametros:
        x : coordenada adimensional x/c (0 a 1)
        f : curvatura maxima como fraccion de la cuerda (ej: 0.02 para 2%)
        xf : posicion de la curvatura maxima como fraccion (ej: 0.40 para 40%)

    Retorna:
        zc : ordenada z de la linea media
        dzc_dx: derivada de la linea media dz_c/dx

    Formulas (Tema 3, diap. 9 / Guion diap. 9):
        Para 0 <= x <= xf: zc = (f/xf^2) * (2*xf*x - x^2)
        Para xf <= x <= 1: zc = (f/(1-xf)^2) * [(1-2*xf) + 2*xf*x - x^2]
    """
    # Caso sin curvatura (perfil simetrico)
    if f == 0 or xf == 0:
        return np.zeros_like(x), np.zeros_like(x)

    zc = np.where(
        x <= xf,
        (f / xf**2) * (2.0 * xf * x - x**2),
        (f / (1.0 - xf)**2) * ((1.0 - 2.0 * xf) + 2.0 * xf * x - x**2)
    )
    )
```



```

dzc_dx = np.where(
    x <= xf,
    (2.0 * f / xf**2) * (xf - x),
    (2.0 * f / (1.0 - xf)**2) * (xf - x)
)
return zc, dzc_dx

def espesor_naca4(x, t):
    """
    Distribucion de espesor para perfiles NACA de 4 cifras (plano xz).

    Parametros:
        x : coordenada adimensional x/c
        t : espesor maximo como fraccion de la cuerda (ej: 0.12 para 12%)

    Retorna:
        zt: semi-espesor en la posicion x (se suma/resta en z a la linea media)

    Formula (Tema 3, diap. 9 / Guion diap. 9):
        zt(x) = +/-5t*(a*raiz(x) + b*x + c*x^2 + d*x^3 + e*x^4)
        con a=0.2969, b=-0.1260, c=-0.3516, d=0.2843, e=-0.1015
    """
    # Coeficientes estandar NACA
    a, b, c, d, e = 0.2969, -0.1260, -0.3516, 0.2843, -0.1015

    zt = 5.0 * t * (a * np.sqrt(np.maximum(x, 0.0)) + b * x + c * x**2
                    + d * x**3 + e * x**4)
    return zt

def generar_perfil_naca4(f, xf, t, N):
    """
    Genera las coordenadas de un perfil NACA de 4 cifras con N paneles.

    Panelizacion con distribucion coseno (concentra nodos en BA y BS).
    El recorrido va desde el Borde de Salida (BS) por el INTRADOS,
    pasando por el Borde de Ataque (BA), y volviendo por el EXTRADOS al BS.

    Parametros:
        f : curvatura maxima (fraccion de cuerda)
        xf : posicion de curvatura maxima (fraccion de cuerda)
        t : espesor maximo (fraccion de cuerda)
        N : numero de paneles (N+1 nodos)

    Retorna:
        x_nodos, z_nodos: arrays de N+1 coordenadas

    Distribucion coseno (Tema 3, diap. 87):
        beta_i = beta_1 + i*Delta_beta, con Delta_beta = 2*pi/N, beta_1 = 0
    """

```

```

    x_i = (c/2)*(1 + cos(beta_i))

    beta = 0    -> x = c  (BS)
    beta = pi   -> x = 0  (BA)
    beta = 2*pi -> x = c  (BS)
"""
N1 = N + 1 # Numero de nodos
beta = np.linspace(0, 2.0 * np.pi, N1)

# Coordinadas x a lo largo de la cuerda (distribucion coseno)
x_cos = 0.5 * (1.0 + np.cos(beta))

x_nodos = np.zeros(N1)
z_nodos = np.zeros(N1)

for i in range(N1):
    xn = np.clip(x_cos[i], 0.0, 1.0)

    # Linea media y su derivada (en el plano xz)
    if f == 0 or xf == 0:
        zc, dzc = 0.0, 0.0
    else:
        if xn <= xf:
            zc = (f / xf**2) * (2.0 * xf * xn - xn**2)
            dzc = (2.0 * f / xf**2) * (xf - xn)
        else:
            zc = (f / (1.0 - xf)**2) * ((1.0 - 2.0 * xf) + 2.0 * xf * xn - xn**2)
            dzc = (2.0 * f / (1.0 - xf)**2) * (xf - xn)

    # Semi-espesor (perpendicular a la linea media)
    zt = 5.0 * t * (0.2969 * np.sqrt(max(xn, 0.0)) - 0.1260 * xn
                    - 0.3516 * xn**2 + 0.2843 * xn**3 - 0.1015 * xn**4)

    # Angulo local de la linea media
    theta_c = np.arctan(dzc)

    if beta[i] <= np.pi:
        # INTRADOS (lower surface): desde BS hacia BA
        x_nodos[i] = xn + zt * np.sin(theta_c)
        z_nodos[i] = zc - zt * np.cos(theta_c)
    else:
        # EXTRADOS (upper surface): desde BA hacia BS
        x_nodos[i] = xn - zt * np.sin(theta_c)
        z_nodos[i] = zc + zt * np.cos(theta_c)

return x_nodos, z_nodos

# =====
# SECCION 2: METODO DE PANELES

```

```

# (Distribucion de manantiales q_j + torbellinos gamma constantes)
# =====

def metodo_paneles(x_nodos, z_nodos, alpha_deg, V_inf=1.0):
    """
    Resuelve el flujo potencial alrededor del perfil usando el metodo de paneles
    con distribucion de manantiales (q_j) y torbellinos (gamma) de intensidad constante.

    METODOLOGIA (Tema 3, diapositivas 87-97):

    PASO 1 - Geometria de paneles:
        - Punto de control: centro de cada panel (xc_i, zc_i)
        - Inclinacion: theta_i = arctan(Delta_z_i / Delta_x_i)
        - Normal: n_i = (-sin theta_i, cos theta_i) [apunta hacia FUERA]
        - Tangente: tau_i = (cos theta_i, sin theta_i)

    PASO 2 - Coeficientes de influencia (diap. 89-93):
        Para el par (i, j), se calculan en ejes LOCALES del panel j:

        Coordenadas locales del p.c. del panel i:
            x_ij = (xc_i - x_j)*cos theta_j + (zc_i - z_j)*sin theta_j
            z_ij = -(xc_i - x_j)*sin theta_j + (zc_i - z_j)*cos theta_j

        Distancias y angulos:
            r1^2 = x_ij^2 + z_ij^2
            r2^2 = (x_ij - c_j)^2 + z_ij^2
            theta1 = arctan(z_ij / x_ij)
            theta2 = arctan(z_ij / (x_ij - c_j))

        Coeficientes adimensionales en ejes locales (diap. 92):

            Manantiales:                Torbellinos:
            u*_q = -(1/2*pi)*ln(r2/r1)    u*_gamma = (1/2*pi)*(theta2 - theta1)
            w*_q = (1/2*pi)*(theta2 - theta1)    w*_gamma = (1/2*pi)*ln(r2/r1)

        Auto-influencia (i = j):
            u*_q = 0,    w*_q = 1/2                u*_gamma = 1/2,    w*_gamma = 0

    PASO 3 - Rotacion a ejes perfil (diap. 93):
            u_q = u*_q*cos theta_j - w*_q*sin theta_j    (idem para u_gamma)
            w_q = u*_q*sin theta_j + w*_q*cos theta_j    (idem para w_gamma)

    PASO 4 - Proyecciones normal y tangencial (diap. 94):
            V_n = u*(-sin theta_i) + w*cos theta_i    [normal al panel i]
            V_tau = u*cos theta_i + w*sin theta_i    [tangente al panel i]

    PASO 5 - Sistema de ecuaciones (diap. 94-95):
        N ecuaciones geometricas: Suma_j a_ij*q_j + a_{i,N+1}*gamma = I_i
        1 ecuacion de Kutta: V_tau,1 + V_tau,N = 0 (panel 1 y panel N en el BS)
    """

```

```

    donde I_i = V_inf*sin(theta_i - alpha)

Parametros:
    x_nodos, z_nodos : coordenadas de los N+1 nodos
    alpha_deg        : angulo de ataque en grados
    V_inf            : velocidad del flujo libre (por defecto 1.0)

Retorna:
    Cl, Cm_c4, Cp, vel_tang, xc, zc, gamma, q, Cd, Cl_i
    (todo en el plano xz)
"""
alpha = np.radians(alpha_deg)
N = len(x_nodos) - 1 # Numero de paneles

# ---- PASO 1: Geometria de paneles ----
# Punto de control (centro del panel)
xc = 0.5 * (x_nodos[:-1] + x_nodos[1:])
zc = 0.5 * (z_nodos[:-1] + z_nodos[1:])

# Incrementos
dx = x_nodos[1:] - x_nodos[:-1]
dz = z_nodos[1:] - z_nodos[:-1]

# Longitud de cada panel
c_panel = np.sqrt(dx**2 + dz**2)

# Inclination de cada panel (diap. 87): theta_i = arctan(Delta_z/Delta_x)
# NOTA: se usa arctan2 para obtener el cuadrante correcto, -pi <= theta <= pi
theta = np.arctan2(dz, dx)

# ---- PASO 2-4: Coeficientes de influencia ----
# Matrices de influencia:
# An[i,j] : contribucion NORMAL de manantial q_j sobre panel i
# Ang[i]   : contribucion NORMAL de torbellino gamma (acumulada de todos los paneles j)
# At[i,j]  : contribucion TANGENCIAL de manantial q_j sobre panel i
# Atg[i]   : contribucion TANGENCIAL de torbellino gamma (acumulada)

An = np.zeros((N, N))
Ang = np.zeros(N)
At = np.zeros((N, N))
Atg = np.zeros(N)

for i in range(N):
    for j in range(N):
        # -- Coordenadas locales del p.c. del panel i en ejes del panel j --
        # (Tema 3, diap. 88)
        x_ij = ((xc[i] - x_nodos[j]) * np.cos(theta[j])
                + (zc[i] - z_nodos[j]) * np.sin(theta[j]))
        z_ij = (-(xc[i] - x_nodos[j]) * np.sin(theta[j])
                + (zc[i] - z_nodos[j]) * np.cos(theta[j]))

```

```

if i == j:
    # =====
    # AUTO-INFLUENCIA (diap. 92)
    # =====
    # CUIDADO: esta es una de las fuentes de error mas comunes
    # de la IA segun los profesores (error en la diagonal).
    #
    # Manantiales: u*_q_ii = 0,      w*_q_ii = q_i/2 -> u*=0, w*=1/2
    # Torbellinos: u*_gamma_ii = gamma/2,      w*_gamma_ii = 0      -> u*=1/2, w*=0
    # =====

    uq_star = 0.0
    wq_star = 0.5
    ug_star = 0.5
    wg_star = 0.0
else:
    # =====
    # INFLUENCIA CRUZADA (diap. 89, 92)
    # =====
    # Distancias desde el p.c. i a los extremos del panel j
    r1_sq = x_ij**2 + z_ij**2
    r2_sq = (x_ij - c_panel[j])**2 + z_ij**2

    # Proteccion numerica contra log(0)
    r1 = np.sqrt(max(r1_sq, 1e-30))
    r2 = np.sqrt(max(r2_sq, 1e-30))

    # Angulos (diap. 89): theta1 y theta2 medidos desde eje local x'
    # NOTA IMPORTANTE sobre el orden r2/r1 en el logaritmo:
    # El signo del logaritmo es CRUCIAL. Segun la diap. 92:
    #   u*_q = -(1/2*pi)*ln(r2/r1)
    #   w*_gamma = +(1/2*pi)*ln(r2/r1)
    theta1 = np.arctan2(z_ij, x_ij)
    theta2 = np.arctan2(z_ij, x_ij - c_panel[j])

    log_ratio = np.log(r2 / r1)
    dtheta = theta2 - theta1

    # Coeficientes adimensionales en ejes locales (diap. 92)
    uq_star = -(1.0 / (2.0 * np.pi)) * log_ratio      # manantial -> u'
    wq_star = (1.0 / (2.0 * np.pi)) * dtheta          # manantial -> w'
    ug_star = (1.0 / (2.0 * np.pi)) * dtheta          # torbellino -> u'
    wg_star = (1.0 / (2.0 * np.pi)) * log_ratio       # torbellino -> w'

    # ---- PASO 3: Rotacion a ejes globales (perfil) ----
    # (Tema 3, diap. 93)
    cos_j = np.cos(theta[j])
    sin_j = np.sin(theta[j])

    # Velocidad inducida por manantiales, en ejes perfil

```

```

uq_global = uq_star * cos_j - wq_star * sin_j
wq_global = uq_star * sin_j + wq_star * cos_j

# Velocidad inducida por torbellinos, en ejes perfil
ug_global = ug_star * cos_j - wg_star * sin_j
wg_global = ug_star * sin_j + wg_star * cos_j

# ---- PASO 4: Proyecciones normal y tangencial ----
# Normal: n_i = (-sin theta_i, cos theta_i)
# Tangente: tau_i = (cos theta_i, sin theta_i)
sin_i = np.sin(theta[i])
cos_i = np.cos(theta[i])

# Contribucion NORMAL del panel j (manantiales y torbellinos)
An[i, j] = -uq_global * sin_i + wq_global * cos_i
Ang[i] += -ug_global * sin_i + wg_global * cos_i

# Contribucion TANGENCIAL del panel j (manantiales y torbellinos)
At[i, j] = uq_global * cos_i + wq_global * sin_i
Atg[i] += ug_global * cos_i + wg_global * sin_i

# ---- PASO 5: Ensamblaje del sistema de ecuaciones ----
# Tamano (N+1) x (N+1)
A_sis = np.zeros((N + 1, N + 1))
b_sis = np.zeros(N + 1)

# --- Primeras N filas: condicion geometrica (V*n = 0) ---
# Suma_j An[i,j]*q_j + Ang[i]*gamma = I_i
# donde I_i = V_inf*sin(theta_i - alpha) [proyeccion de V_inf sobre la normal, cambiada de
# signo]
A_sis[:N, :N] = An # Coeficientes de q_j
A_sis[:N, N] = Ang # Coeficiente de gamma
b_sis[:N] = V_inf * np.sin(theta - alpha) # Terminos independientes I_i

# --- Fila N+1: Condicion de Kutta (diap. 94) ---
# La presion debe ser igual en extrados e intrados en el BS.
# Esto se traduce en: V_tau,1 + V_tau,N = 0
# (velocidades tangenciales en el primer y ultimo panel suman cero)
#
# V_tau,i = Suma_j At[i,j]*q_j + Atg[i]*gamma + V_inf*cos(theta_i - alpha)
#
# Para panel 1 (i=0) y panel N (i=N-1):
A_sis[N, :N] = At[0, :] + At[N-1, :]
A_sis[N, N] = Atg[0] + Atg[N-1]
b_sis[N] = -V_inf * (np.cos(theta[0] - alpha) + np.cos(theta[N-1] - alpha))

# ---- Resolucion del sistema lineal ----
solucion = np.linalg.solve(A_sis, b_sis)
q = solucion[:N] # Intensidades de los manantiales (una por panel)
gamma = solucion[N] # Intensidad del torbellino (unica para todos los paneles)

```

```

# ---- Calculo de la velocidad tangencial en cada panel ----
# V_tau,i = Suma_j At[i,j]*q_j + Atg[i]*gamma + V_inf*cos(theta_i - alpha)
# (Tema 3, diap. 96)
vel_tang = At @ q + Atg * gamma + V_inf * np.cos(theta - alpha)

# ---- Coeficiente de presion ----
# Cp = 1 - (V_local / V_inf)^2
Cp = 1.0 - (vel_tang / V_inf)**2

# =====
# CALCULO DE Cl, Cm_c/4 - PANEL POR PANEL
# =====
# NOTA IMPORTANTE: El Cl se calcula integrando Cp sobre cada panel
# (fuerza de presion * proyeccion), NO como un unico torbellino
# usando Kutta-Yukovski Gamma = rho*V_inf*Cl*c/2.
# Esto es una fuente de error frecuente senalada por los profesores.
#
# La fuerza de presion sobre el panel i actua en direccion normal:
# dF = -Cp_i * q_inf * c_i * n_i
#
# En componentes del perfil (body axes):
# dCFx_i = Cp_i * Delta_z_i / c (fuerza en x por unidad de q_inf*c)
# dCFz_i = -Cp_i * Delta_x_i / c (fuerza en z por unidad de q_inf*c)
#
# Coeficientes aerodinamicos:
# CN = Suma dCFz = -Suma Cp_i * Delta_x_i / c (fuerza normal, + hacia arriba)
# CA = Suma dCFx = Suma Cp_i * Delta_z_i / c (fuerza axial, + hacia BS)
# Cl = CN*cos alpha - CA*sin alpha
# Cd = CN*sin alpha + CA*cos alpha (~ 0 en flujo potencial)
# =====

c_cuerda = 1.0 # Cuerda unitaria

# Fuerza normal (perpendicular a la cuerda, positiva hacia arriba)
CN = -np.sum(Cp * dx) / c_cuerda

# Fuerza axial (a lo largo de la cuerda, positiva hacia BS)
CA = np.sum(Cp * dz) / c_cuerda

# Coeficiente de sustentacion (perpendicular a V_inf)
Cl = CN * np.cos(alpha) - CA * np.sin(alpha)

# Coeficiente de resistencia de presion (deberia ser ~ 0 en flujo potencial)
Cd = CN * np.sin(alpha) + CA * np.cos(alpha)

# =====
# MOMENTO RESPECTO AL BORDE DE ATAQUE (positivo nariz arriba)
# =====
# Cm_le = Suma_i [-Cp_i * (xc_i * Delta_x_i + zc_i * Delta_z_i)] / c^2

```

```

#
# NOTA: El signo se define cuidadosamente:
# El momento "matematico" (producto cruz) da positivo = nariz abajo.
# La convencion aeronautica usa positivo = nariz arriba.
# Derivacion:
#   M_le (math, CCW+) = Suma [xc_i * dFz_i - zc_i * dFx_i]
#   Cm_le (aero, nose-up+) = -M_le / (q_inf*c^2)
#   = -Suma [xc_i*(-Cp_i*Delta_x_i) - zc_i*(Cp_i*Delta_z_i)] / c^2
#   = Suma Cp_i * (xc_i*Delta_x_i + zc_i*Delta_z_i) / c^2
# =====

Cm_le = np.sum(Cp * (xc * dx + zc * dz)) / c_cuerda**2

# Traslacion al cuarto de cuerda:
# Cm_c/4 = Cm_le + CN * 0.25
# (Tema 3, diap. 6: Cm_B = Cm_A + Cl*(xB - xA)/c, con CN ~ Cl para alpha pequeno)
Cm_c4 = Cm_le + CN * 0.25
Cl_i = (-Cp * dx * np.cos(alpha) - Cp * dz * np.sin(alpha)) / c_cuerda
return Cl, Cm_c4, Cp, vel_tang, xc, zc, gamma, q, Cd, Cl_i

# =====
# SECCION 3: ESTUDIO DE CONVERGENCIA
# =====

def estudio_convergencia(f, xf, t, alpha_deg, N_inicio=40, factor=1.2,
                        eps_objetivo=0.01, N_max=1000):
    """
    Estudio de convergencia segun el criterio del guion (diap. 11):

    1. Comenzar con N_inicio paneles (40).
    2. Calcular Cl_i.
    3. Aumentar paneles un 20% (factor=1.2). Calcular Cl_{i+1}.
    4. Error relativo: eps_i = |Cl_{i+1} - Cl_i| / |Cl_i|
    5. Repetir hasta eps < 1%.

    Retorna:
        N_list      : lista de numeros de paneles usados
        Cl_list     : lista de Cl correspondientes
        eps_list    : lista de errores relativos
        N_opt       : numero de paneles optimo (primer N con eps < 1%)
    """
    N_list = []
    Cl_list = []
    eps_list = []

    N_actual = N_inicio

    # Primera iteracion
    x, z = generar_perfil_naca4(f, xf, t, N_actual)

```

```

Cl_actual, _, _, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)

N_list.append(N_actual)
Cl_list.append(Cl_actual)

N_opt = None

while N_actual < N_max:
    # Aumentar paneles un 20%
    N_nuevo = int(np.ceil(N_actual * factor))
    if N_nuevo == N_actual:
        N_nuevo += 2 # Asegurar incremento (y par para simetria)
    # Hacer que sea par para simetria intrados/extrados
    if N_nuevo % 2 != 0:
        N_nuevo += 1

    x, z = generar_perfil_naca4(f, xf, t, N_nuevo)
    Cl_nuevo, _, _, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)

    # Error relativo
    if abs(Cl_actual) > 1e-10:
        eps = abs(Cl_nuevo - Cl_actual) / abs(Cl_actual)
    else:
        eps = abs(Cl_nuevo - Cl_actual)

    N_list.append(N_nuevo)
    Cl_list.append(Cl_nuevo)
    eps_list.append(eps)

    # Verificar convergencia
    if eps < eps_objetivo and N_opt is None:
        N_opt = N_nuevo

    Cl_actual = Cl_nuevo
    N_actual = N_nuevo

    # Si ya convergio, hacer algunas iteraciones mas para el grafico
    if N_opt is not None and N_actual > N_opt * 2:
        break

if N_opt is None:
    N_opt = N_list[-1]
    print(f" ! No se alcanzo convergencia < {eps_objetivo*100}%. Usando N = {N_opt}")

return N_list, Cl_list, eps_list, N_opt

# =====
# SECCION 4: RETO DE DISEÑO (Apartado 3 del guion)
# =====

```

```

def reto_diseno(f_base, xf_base, t_base, alpha_deg, delta_Cl=0.2, N=200):
    """
    Optimizacion parametrica para incrementar Cl en +0.2 (plano xz),
    manteniendo |Delta_Cm_c/4| lo mas bajo posible.

    Estrategia analitica (Tema 3, diap. 15-16):
        Cl ~ 2*pi*(alpha + 2f/c)    -> f es el DRIVER PRINCIPAL de Cl
        Cm_c/4 ~ -pi*f/c           -> Cm depende SOLO de f
        Cl_alpha ~ 2*pi*(1+0.83t/c) -> t tiene efecto SECUNDARIO
        xf no cambia Cl en flujo potencial (solo cambia forma del Cp)
    """
    # ---- Valores base ----
    x_base, z_base = generar_perfil_naca4(f_base, xf_base, t_base, N)
    Cl_base, Cm_base, _, _, _, _, _, _, _ = metodo_paneles(x_base, z_base, alpha_deg)
    Cl_objetivo = Cl_base + delta_Cl

    print(f"\n{'='*70}")
    print(f"  APARTADO 3: RETO DE DISEÑO E INGENIERIA INVERSA")
    print(f"{'='*70}")
    print(f"  Perfil base: f={f_base*100:.1f}%, xf={xf_base*100:.0f}%, t={t_base*100:.0f}%")
    print(f"  alpha = {alpha_deg} grados")
    print(f"  Cl_base   = {Cl_base:.6f}")
    print(f"  Cm_base   = {Cm_base:.6f}")
    print(f"  Cl_objetivo = {Cl_objetivo:.6f} (Delta_Cl = +{delta_Cl})")

    # =====
    # PASO 1: ANALISIS DE SENSIBILIDAD - Efecto de cada parametro
    # =====
    print(f"\n{'-'*70}")
    print(f"  PASO 1: ANALISIS DE SENSIBILIDAD (un parametro a la vez)")
    print(f"{'-'*70}")

    # --- 1a. Efecto de la curvatura f ---
    print(f"\n  1a) Efecto de la curvatura f (xf={xf_base*100:.0f}% fijo, t={t_base*100:.0f}% fijo):")
    print(f"      TPL predice: dCl/d(f/c) ~ 4*pi ~ 12.57")
    print(f"      {'f (%)':>8s} {'Cl':>10s} {'Delta_Cl':>10s} {'Cm_c/4':>10s} {'Delta|Cm|':>10s}")
    print(f"      {'-'*55}")
    f_scan = np.arange(max(0.5, f_base*100 - 1), f_base*100 + 6, 0.5) / 100.0
    for fv in f_scan:
        x, z = generar_perfil_naca4(fv, xf_base, t_base, N)
        Cl_v, Cm_v, _, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
        marca = " <-- base" if abs(fv - f_base) < 1e-6 else ""
        marca = " <-- OBJETIVO" if abs(Cl_v - Cl_objetivo) < 0.015 else marca
        print(f"      {fv*100:8.1f} {Cl_v:10.6f} {Cl_v-Cl_base:+10.4f} {Cm_v:10.6f} {abs(Cm_v)-abs(Cm_base):+10.4f}{marca}")

    # --- 1b. Efecto de la posicion de curvatura xf ---

```

```

print(f"\n 1b) Efecto de la posicion xf (f={f_base*100:.1f}% fijo, t={t_base*100:.0f}% fijo):
")
print(f"      TPL predice: Cl NO depende de xf -> efecto pequeno")
print(f"      {'xf (%)':>8s} {'Cl':>10s} {'Delta_Cl':>10s} {'Cm_c/4':>10s} {'Delta|Cm|':>10s}")
print(f"      {'-'*55}")
xf_scan = np.arange(20, 70, 10) / 100.0
for xfv in xf_scan:
    x, z = generar_perfil_naca4(f_base, xfv, t_base, N)
    Cl_v, Cm_v, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
    marca = " <-- base" if abs(xfv - xf_base) < 1e-6 else ""
    print(f"      {xfv*100:8.0f} {Cl_v:10.6f} {Cl_v-Cl_base:+10.4f} {Cm_v:10.6f} {abs(Cm_v)-abs(Cm_base):+10.4f}{marca}")

# --- 1c. Efecto del espesor t ---
print(f"\n 1c) Efecto del espesor t (f={f_base*100:.1f}% fijo, xf={xf_base*100:.0f}% fijo):")
print(f"      TPL predice: Cl_alpha ~ 2*pi*(1+0.83*t/c) -> efecto secundario")
print(f"      {'t (%)':>8s} {'Cl':>10s} {'Delta_Cl':>10s} {'Cm_c/4':>10s} {'Delta|Cm|':>10s}")
print(f"      {'-'*55}")
t_scan = np.arange(max(6, t_base*100 - 4), t_base*100 + 8, 2) / 100.0
for tv in t_scan:
    x, z = generar_perfil_naca4(f_base, xf_base, tv, N)
    Cl_v, Cm_v, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
    marca = " <-- base" if abs(tv - t_base) < 1e-6 else ""
    print(f"      {tv*100:8.0f} {Cl_v:10.6f} {Cl_v-Cl_base:+10.4f} {Cm_v:10.6f} {abs(Cm_v)-abs(Cm_base):+10.4f}{marca}")

# --- Conclusion del analisis de sensibilidad ---
print(f"\n CONCLUSION DE SENSIBILIDAD:")
print(f"      -> f (curvatura) es el parametro DOMINANTE: Delta_Cl ~ 12*Delta(f/c)")
print(f"      -> xf (posicion) apenas cambia Cl en flujo potencial")
print(f"      -> t (espesor) tiene efecto pequeno a traves del factor (1+0.83*t/c)")
print(f"      -> Para Delta_Cl = +0.2: Delta_f/c ~ 0.2/(4*pi) ~ 0.016 -> f_nueva ~ {f_base*100:.1f}+1.6 = {f_base*100+1.6:.1f}%")

# =====
# PASO 2: BARRIDO PARAMETRICO FINO
# =====
print(f"\n{'-'*70}")
print(f" PASO 2: BARRIDO PARAMETRICO FINO")
print(f"{'-'*70}")

# Rangos : f fino (es el driver), xf y t solo 3 valores cada uno
f_range = np.arange(max(0.5, f_base*100), f_base*100 + 5, 0.1) / 100.0
xf_range = np.array([max(10, xf_base*100 - 10), xf_base*100, min(90, xf_base*100 + 10)]) / 100.0
t_range = np.array([t_base*100, t_base*100 + 1, t_base*100 + 2]) / 100.0

n_total = len(f_range) * len(xf_range) * len(t_range)

```

```

print(f" Combinaciones a evaluar: {len(f_range)} f x {len(xf_range)} xf x {len(t_range)} t = {
n_total}")

# Usar pocos paneles para la busqueda (rapido), luego verificar con N completo
N_busqueda = 80
print(f" Paneles en busqueda: {N_busqueda} (se verifica despues con N={N})")

mejor_resultado = None
mejor_delta_cm = np.inf
resultados = []
n_eval = 0

for t_val in t_range:
    for xf_val in xf_range:
        for f_val in f_range:
            n_eval += 1
            if n_eval % 200 == 0:
                print(f" ... evaluando {n_eval}/{n_total}")
            x, z = generar_perfil_naca4(f_val, xf_val, t_val, N_busqueda)
            Cl, Cm, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)

            if abs(Cl - Cl_objetivo) < 0.01:
                delta_cm = abs(Cm - Cm_base)
                resultados.append({
                    'f': f_val, 'xf': xf_val, 't': t_val,
                    'Cl': Cl, 'Cm': Cm, 'delta_Cm': delta_cm
                })
                if delta_cm < mejor_delta_cm:
                    mejor_delta_cm = delta_cm
                    mejor_resultado = {
                        'f': f_val, 'xf': xf_val, 't': t_val,
                        'Cl': Cl, 'Cm': Cm, 'delta_Cm': delta_cm
                    }

if mejor_resultado is None:
    print(" ! No se encontro solucion con |Delta_Cl| < 0.01. Buscando la mas cercana...")
    mejor_delta_cl = np.inf
    for t_val in t_range:
        for xf_val in xf_range:
            for f_val in f_range:
                x, z = generar_perfil_naca4(f_val, xf_val, t_val, N_busqueda)
                Cl, Cm, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
                delta_cl_abs = abs(Cl - Cl_objetivo)
                delta_cm = abs(Cm - Cm_base)
                if delta_cl_abs < mejor_delta_cl:
                    mejor_delta_cl = delta_cl_abs
                    mejor_resultado = {
                        'f': f_val, 'xf': xf_val, 't': t_val,
                        'Cl': Cl, 'Cm': Cm, 'delta_Cm': delta_cm
                    }

```

```

# VERIFICACION: recalculamos el optimo y las top soluciones con N paneles completo
print(f"\n Verificando mejores candidatos con N={N} paneles...")
for r in resultados:
    x, z = generar_perfil_naca4(r['f'], r['xf'], r['t'], N)
    Cl, Cm, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
    r['Cl'] = Cl
    r['Cm'] = Cm
    r['delta_Cm'] = abs(Cm - Cm_base)

# Recalcular el mejor con N completo
x, z = generar_perfil_naca4(mejor_resultado['f'], mejor_resultado['xf'], mejor_resultado['t'],
N)
Cl, Cm, _, _, _, _, _, _ = metodo_paneles(x, z, alpha_deg)
mejor_resultado['Cl'] = Cl
mejor_resultado['Cm'] = Cm
mejor_resultado['delta_Cm'] = abs(Cm - Cm_base)

# =====
# PASO 3: RESULTADOS
# =====

print(f"\n{'-'*70}")
print(f" PASO 3: RESULTADOS DEL RETO DE DISEÑO")
print(f"{'-'*70}")

print(f"\n Soluciones que cumplen |Cl - Cl_obj| < 0.01: {len(resultados)}")
if len(resultados) > 0:
    # Mostrar las 10 mejores
    resultados_ord = sorted(resultados, key=lambda r: r['delta_Cm'])
    n_mostrar = min(10, len(resultados_ord))
    print(f"\n Top {n_mostrar} soluciones (ordenadas por menor |Delta_Cm|):")
    print(f"  {'#':>3s}  {'f(%)':>6s}  {'xf(%)':>6s}  {'t(%)':>6s}  {'Cl':>10s}  {'Cm_c/4':>10s}  {'Delta_Cl':>8s}  {'Delta|Cm|':>8s}")
    print(f"{'-'*68}")
    for k, r in enumerate(resultados_ord[:n_mostrar]):
        print(f"  {k+1:3d}  {r['f']*100:6.1f}  {r['xf']*100:6.0f}  {r['t']*100:6.0f}  "
              f"{r['Cl']:10.6f}  {r['Cm']:10.6f}  {r['Cl']-Cl_base:+8.4f}  {r['delta_Cm']:8.4f}
")

f_opt = mejor_resultado['f']
xf_opt = mejor_resultado['xf']
t_opt = mejor_resultado['t']

print(f"\n -----")
print(f"          PERFIL OPTIMIZADO SELECCIONADO          ")
print(f" -----")
print(f"  f/c  = {f_opt*100:5.1f}%  (base: {f_base*100:.1f})%  ")
print(f"  xf   = {xf_opt*100:5.0f}%  (base: {xf_base*100:.0f})%  ")
print(f"  t/c  = {t_opt*100:5.0f}%  (base: {t_base*100:.0f})%  ")
print(f"  Cl   = {mejor_resultado['Cl']:.6f}  (Delta_Cl = {mejor_resultado['Cl']-Cl_base:+.4f}%)

```

```

f}) ")
print(f"      Cm_c/4= {mejor_resultado['Cm']:.6f} (Delta|Cm|= {mejor_resultado['delta_Cm']:.4f})
      ")
print(f" -----")

# --- Explicacion fisica de por que varia cada parametro ---
print(f"\n  ANALISIS CRITICO (Por que estos valores?):")
print(f" -----")

delta_f = f_opt - f_base
delta_xf = xf_opt - xf_base
delta_t = t_opt - t_base

print(f"\n  * Curvatura f: {f_base*100:.1f}% -> {f_opt*100:.1f}% (Delta_f = {delta_f*100:+.1f}
  %)")
print(f"      Es el cambio principal. En TPL: Cl_0 = 4*pi*f/c. Aumentar f")
print(f"      incrementa directamente la sustentacion a angulo de ataque cero")
print(f"      porque la linea media mas curvada genera mayor asimetria entre")
print(f"      extrados e intrados, acelerando el flujo arriba y decelerandolo abajo.")
print(f"      Coste: Cm_c/4 = -pi*f/c, asi que mas curvatura -> mas momento picador.")

if abs(delta_xf) > 1e-6:
    print(f"\n  * Posicion xf: {xf_base*100:.0f}% -> {xf_opt*100:.0f}% (Delta_xf = {delta_xf
    *100:+.0f}%)")
    print(f"      En flujo potencial, xf no cambia Cl de forma significativa.")
    print(f"      Su efecto principal es redistribuir la carga de presion a lo largo")
    print(f"      de la cuerda: xf mas adelante concentra la succion cerca del BA.")
    print(f"      En flujo viscoso, esto si importa: afecta al gradiente de presion")
    print(f"      adverso y por tanto a la posicion de desprendimiento y al Cl_max.")
else:
    print(f"\n  * Posicion xf: se mantiene en {xf_opt*100:.0f}%")
    print(f"      Coherente con TPL: xf no afecta a Cl en flujo potencial.")

if abs(delta_t) > 1e-6:
    print(f"\n  * Espesor t: {t_base*100:.0f}% -> {t_opt*100:.0f}% (Delta_t = {delta_t*100:+.0
    f}%)")
    print(f"      El espesor afecta a Cl a traves del factor (1+0.83*t/c) que multiplica")
    print(f"      a 2*pi*alpha. A mayor espesor, mas sustentacion para un alpha dado, pero el
    efecto")
    print(f"      es pequeno comparado con cambiar f. No afecta a Cm_c/4 significativamente.")
else:
    print(f"\n  * Espesor t: se mantiene en {t_opt*100:.0f}%")
    print(f"      Confirma que el espesor no es un driver relevante para Delta_Cl en potencial.")

return f_opt, xf_opt, t_opt, mejor_resultado, Cl_base, Cm_base, resultados

# =====
# SECCION 5: EXPORTACION DE COORDENADAS PARA XFOIL
# =====

```

```

def exportar_para_xfoil(x_nodos, z_nodos, nombre_archivo="perfil_optimizado.dat"):
    """
    Exporta las coordenadas del perfil en formato compatible con XFOIL.
    XFOIL espera: coordenadas desde BS por extrados, BA, intrados, BS.
    Nuestro formato va: BS -> intrados -> BA -> extrados -> BS.
    Hay que invertir el orden.
    """
    # Invertir: XFOIL usa extrados -> BA -> intrados
    x_xfoil = x_nodos[::-1]
    z_xfoil = z_nodos[::-1]

    with open(nombre_archivo, 'w') as f:
        f.write(f"Perfil NACA optimizado\n")
        for i in range(len(x_xfoil)):
            f.write(f" {x_xfoil[i]:.6f} {z_xfoil[i]:.6f}\n")

    print(f" Coordenadas exportadas a: {nombre_archivo}")
    return nombre_archivo

# =====
# SECCION 6: GRAFICOS DE CALIDAD
# =====

def configurar_grafico():
    """Configuracion global de matplotlib para graficos de calidad profesional."""
    plt.rcParams.update({
        'font.size': 11,
        'font.family': 'serif',
        'axes.labelsize': 12,
        'axes.titlesize': 13,
        'legend.fontsize': 10,
        'xtick.labelsize': 10,
        'ytick.labelsize': 10,
        'figure.dpi': 150,
        'savefig.dpi': 300,
        'axes.grid': True,
        'grid.alpha': 0.3,
        'lines.linewidth': 1.5,
    })

configurar_grafico()

def graficar_perfil(x, z, titulo="Perfil NACA", archivo=None):
    """Dibuja el perfil aerodinamico con ejes proporcionados."""
    fig, ax = plt.subplots(figsize=(10, 3))
    ax.plot(x, z, 'b-', linewidth=1.5)
    ax.fill(x, z, alpha=0.1, color='blue')

```

```

ax.set_xlabel('x/c')
ax.set_ylabel('z/c')
ax.set_title(titulo)
ax.set_aspect('equal')
ax.set_xlim(-0.05, 1.05)
ax.grid(True, alpha=0.3)
plt.tight_layout()
if archivo:
    plt.savefig(archivo, bbox_inches='tight')
plt.show()

def graficar_cp(xc, Cp, titulo="Distribucion de Cp", archivo=None,
               xc2=None, Cp2=None, label1="Panel Method", label2="XFoil"):
    """Dibuja la distribucion de Cp (eje y invertido, convencion aeronautica)."""
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(xc, Cp, 'b-o', markersize=2, label=label1)
    if xc2 is not None and Cp2 is not None:
        ax.plot(xc2, Cp2, 'r--s', markersize=2, label=label2)
        ax.legend()
    ax.set_xlabel('x/c')
    ax.set_ylabel('Cp')
    ax.set_title(titulo)
    ax.invert_yaxis() # Convencion: Cp negativo arriba
    ax.grid(True, alpha=0.3)
    ax.axhline(y=0, color='k', linewidth=0.5)
    plt.tight_layout()
    if archivo:
        plt.savefig(archivo, bbox_inches='tight')
    plt.show()

def graficar_convergencia(N_list, Cl_list, eps_list, N_opt, archivo=None):
    """
    Grafico de Cl(N) y eps(N) con doble escala (apartado 2.4 del guion).
    """
    fig, ax1 = plt.subplots(figsize=(10, 6))

    color1 = 'tab:blue'
    ax1.set_xlabel('Numero de paneles (N)')
    ax1.set_ylabel('Cl', color=color1)
    ax1.plot(N_list, Cl_list, 'o-', color=color1, label='Cl(N)')
    ax1.tick_params(axis='y', labelcolor=color1)
    ax1.axvline(x=N_opt, color='green', linestyle='--', alpha=0.7, label=f'N_opt = {N_opt}')

    ax2 = ax1.twinx()
    color2 = 'tab:red'
    ax2.set_ylabel('Error relativo eps (%)', color=color2)
    # eps_list tiene un elemento menos que N_list
    ax2.plot(N_list[1:], [e * 100 for e in eps_list], 's--', color=color2, label='eps(N) [%]')

```

```

ax2.axhline(y=1.0, color='red', linestyle=':', alpha=0.5, label='Objetivo: eps = 1%')
ax2.tick_params(axis='y', labelcolor=color2)
ax2.set_yscale('log')

# Leyenda combinada
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='center right')

ax1.set_title('Estudio de Convergencia: C1(N) y Error Relativo eps(N)')
fig.tight_layout()
if archivo:
    plt.savefig(archivo, bbox_inches='tight')
plt.show()

def graficar_comparacion_perfiles(x1, z1, x2, z2, label1, label2, archivo=None):
    """Dibuja dos perfiles superpuestos (apartado 3.2)."""
    fig, ax = plt.subplots(figsize=(10, 3.5))
    ax.plot(x1, z1, 'b-', linewidth=1.5, label=label1)
    ax.plot(x2, z2, 'r--', linewidth=1.5, label=label2)
    ax.set_xlabel('x/c')
    ax.set_ylabel('z/c')
    ax.set_title('Comparacion de perfiles')
    ax.set_aspect('equal')
    ax.set_xlim(-0.05, 1.05)
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    if archivo:
        plt.savefig(archivo, bbox_inches='tight')
    plt.show()

def graficar_cp_comparacion(xc1, Cp1, xc2, Cp2, label1, label2, archivo=None):
    """Dibuja dos distribuciones de Cp superpuestas (apartado 3.3)."""
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.plot(xc1, Cp1, 'b-o', markersize=2, label=label1)
    ax.plot(xc2, Cp2, 'r--s', markersize=2, label=label2)
    ax.set_xlabel('x/c')
    ax.set_ylabel('Cp')
    ax.set_title('Comparacion de distribuciones de Cp')
    ax.invert_yaxis()
    ax.axhline(y=0, color='k', linewidth=0.5)
    ax.legend()
    ax.grid(True, alpha=0.3)
    plt.tight_layout()
    if archivo:
        plt.savefig(archivo, bbox_inches='tight')
    plt.show()

```

```

# =====
# SECCION 7: EJECUCION PRINCIPAL
# =====

def main():
    """
    Ejecucion completa de los apartados 2 y 3 del trabajo.

    ! IMPORTANTE: Modifica las variables f, xf, t y alpha segun tu DNI.
    Las formulas del guion (diap. 8) son:

    Para DNI: 12345XYZ
    Perfil NACA:
        f  = 1 + E(X/8) * 3      [en % de cuerda]
        xf = 20 + 10*E(Y/8) * 2  [en % de cuerda]
        t  = 10 + E(Z/9) * 8     [en % de cuerda]

    Perfil triangular:
        alpha = (1 + 0.3*X) grados
        xv    = 0.25 + 0.05*Y
        zv    = 0.1 + 0.01*Z

    donde E(x) es la parte entera de x.

    EJEMPLO: DNI 12345678 -> X=6, Y=7, Z=8
        f  = 1 + E(6/8)*3 = 1 + 0*3 = 1%
        xf = 20 + 10*E(7/8)*2 = 20 + 10*0*2 = 20%
        t  = 10 + E(8/9)*8 = 10 + 0*8 = 10%
        -> NACA 1210
    """

    print("=" * 70)
    print("  METODO DE PANELES - PERFILES NACA DE 4 CIFRAS")
    print("  Distribucion de manantiales (qj) + torbellinos (gamma) constantes")
    print("=" * 70)

    # =====
    # PARAMETROS DEL PERFIL - MODIFICAR SEGUN TU DNI
    # =====
    f  = 0.03    # Curvatura maxima (fraccion de cuerda)
    xf = 0.3     # Posicion de curvatura maxima (fraccion de cuerda)
    t  = 0.16    # Espesor maximo (fraccion de cuerda)

    # Angulo de ataque del perfil triangular (se usa tambien para el NACA)
    # alpha = (1 + 0.3*X) grados -> Modificar segun DNI
    alpha_deg = 2.8 # Ejemplo para X=6: alpha = 1 + 0.3*6 = 2.8 grados

    nombre_perfil = f"NACA {int(f*100)}{int(xf*10)}{int(t*100):02d}"

```

```

print(f"\n Perfil: {nombre_perfil}")
print(f" f = {f*100:.1f}%, xf = {xf*100:.0f}%, t = {t*100:.0f}%")
print(f" alpha = {alpha_deg} grados\n")

# =====
# APARTADO 2.1: Codigo de paneles con 100 paneles iniciales
# =====

print("-" * 60)
print(" APARTADO 2.1: Implementacion con 100 paneles")
print("-" * 60)

N_inicial = 100
x_100, z_100 = generar_perfil_naca4(f, xf, t, N_inicial)
Cl_100, Cm_100, Cp_100, vel_100, xc_100, zc_100, gamma_100, q_100, Cd_100, Cl_i_100 = \
metodo_paneles(x_100, z_100, alpha_deg)

print(f" N = {N_inicial} paneles:")
print(f" Cl = {Cl_100:.6f}")
print(f" Cm_c/4 = {Cm_100:.6f}")
print(f" Cd = {Cd_100:.6f} (deberia ser ~ 0)")
print(f" gamma = {gamma_100:.6f}")

# Dibujar el perfil
graficar_perfil(x_100, z_100,
                titulo=f"Perfil {nombre_perfil} ({N_inicial} paneles)",
                archivo="fig_perfil_base.png")

# =====
# APARTADO 2.2: Comparacion con XFOIL no viscoso
# =====

print(f"\n{'-'*60}")
print(f" APARTADO 2.2: Comparacion con XFOIL (valores de referencia)")
print(f"{'-'*60}")
print(f" -> Ejecuta XFOIL con el perfil {nombre_perfil} a alpha = {alpha_deg} grados")
print(f" en modo NO viscoso (inviscid) y anota Cl y Cm_c/4.")
print(f" -> Introduce aqui los valores de XFOIL para calcular el error %.")

# Valores de XFOIL (REEMPLAZAR con tus datos):
Cl_xfoil = None # Ej: 0.7523
Cm_xfoil = None # Ej: -0.0532

if Cl_xfoil is not None:
    error_Cl = abs(Cl_100 - Cl_xfoil) / abs(Cl_xfoil) * 100
    print(f"\n Cl (codigo) = {Cl_100:.6f}")
    print(f" Cl (XFOIL) = {Cl_xfoil:.6f}")
    print(f" Error Cl = {error_Cl:.2f}%")
if Cm_xfoil is not None:
    error_Cm = abs(Cm_100 - Cm_xfoil) / abs(Cm_xfoil) * 100
    print(f" Cm (codigo) = {Cm_100:.6f}")

```

```

    print(f"   Cm (XFOIL)   = {Cm_xfoil:.6f}")
    print(f"   Error Cm     = {error_Cm:.2f}%")

# =====
# APARTADO 2.3: Distribucion de Cp
# =====
print(f"\n{'-'*60}")
print(f"   APARTADO 2.3: Distribucion de Cp(x)")
print(f"{'-'*60}")

graficar_cp(xc_100, Cp_100,
            titulo=f"Distribucion de Cp - {nombre_perfil} a alpha = {alpha_deg} grados (N = {
N_inicial})",
            archivo="fig_cp_100paneles.png")

# =====
# APARTADO 2.4-2.5: Estudio de convergencia
# =====
print(f"\n{'-'*60}")
print(f"   APARTADO 2.4-2.5: Estudio de convergencia")
print(f"{'-'*60}")

N_list, Cl_list, eps_list, N_opt = estudio_convergencia(
    f, xf, t, alpha_deg, N_inicio=40, factor=1.2, eps_objetivo=0.01
)

print(f"\n Resultados del estudio de convergencia:")
print(f"   {'N':>6s}   {'Cl':>12s}   {'eps (%)':>10s}")
print(f"   {'-'*30}")
for k in range(len(N_list)):
    if k == 0:
        print(f"   {N_list[k]:6d}   {Cl_list[k]:12.6f}   {'---':>10s}")
    else:
        print(f"   {N_list[k]:6d}   {Cl_list[k]:12.6f}   {eps_list[k-1]*100:10.4f}")

print(f"\n OK Numero de paneles optimo: N_opt = {N_opt}")

graficar_convergencia(N_list, Cl_list, eps_list, N_opt,
                      archivo="fig_convergencia.png")

# Recalcular con N_opt
x_opt, z_opt = generar_perfil_naca4(f, xf, t, N_opt)
Cl_opt, Cm_opt, Cp_opt, vel_opt, xc_opt, zc_opt, _, _, Cd_opt, _ = \
metodo_paneles(x_opt, z_opt, alpha_deg)

print(f"\n Resultados con N_opt = {N_opt}:")
print(f"   Cl       = {Cl_opt:.6f}")
print(f"   Cm_c/4   = {Cm_opt:.6f}")
print(f"   Cd       = {Cd_opt:.6f}")

```

```

graficar_cp(xc_opt, Cp_opt,
            titulo=f"Distribucion de Cp - {nombre_perfil} a alpha = {alpha_deg} grados (N_opt =
            {N_opt})",
            archivo="fig_cp_Nopt.png")

# =====
# APARTADO 3: RETO DE DISEÑO (la funcion reto_diseno imprime todo)
# =====

N_diseno = min(N_opt, 200)

f_new, xf_new, t_new, resultado, Cl_base, Cm_base, todos_resultados = \
    reto_diseno(f, xf, t, alpha_deg, delta_Cl=0.2, N=N_diseno)

# Generar el perfil optimizado para graficos
x_new, z_new = generar_perfil_naca4(f_new, xf_new, t_new, N_diseno)
Cl_new, Cm_new, Cp_new, vel_new, xc_new, zc_new, _, _, _, _ = \
    metodo_paneles(x_new, z_new, alpha_deg)

nombre_nuevo = f"Modificado (f={f_new*100:.1f}%, xf={xf_new*100:.0f}%, t={t_new*100:.0f}%)"

# Apartado 3.2: Comparacion de perfiles
x_base_plot, z_base_plot = generar_perfil_naca4(f, xf, t, N_diseno)
graficar_comparacion_perfiles(
    x_base_plot, z_base_plot, x_new, z_new,
    label1=nombre_perfil, label2=nombre_nuevo,
    archivo="fig_comparacion_perfiles.png"
)

# Apartado 3.3: Comparacion de Cp
Cl_b, Cm_b, Cp_b, _, xc_b, _, _, _, _ = metodo_paneles(x_base_plot, z_base_plot, alpha_deg)
graficar_cp_comparacion(
    xc_b, Cp_b, xc_new, Cp_new,
    label1=nombre_perfil, label2=nombre_nuevo,
    archivo="fig_comparacion_cp.png"
)

# =====
# EXPORTAR PERFIL OPTIMIZADO PARA XFOIL (Apartado 4)
# =====
print(f"\n{'-'*60}")
print(f"  EXPORTACION para XFOIL (Apartado 4)")
print(f"{'-'*60}")

# Generar con muchos paneles para exportacion suave
x_export, z_export = generar_perfil_naca4(f_new, xf_new, t_new, 200)
exportar_para_xfoil(x_export, z_export, "perfil_optimizado.dat")

# Tambien exportar el perfil base
x_exp_base, z_exp_base = generar_perfil_naca4(f, xf, t, 200)

```

```

exportar_para_xfoil(x_exp_base, z_exp_base, "perfil_base.dat")

print(f"\n Para el apartado 4, en XFOIL:")
print(f" 1. Cargar 'perfil_optimizado.dat' con LOAD")
print(f" 2. OPER -> VISC 3000000 (Re = 3*10^6)")
print(f" 3. ASEQ alphamin alphamax Delta_alpha")
print(f" 4. Comparar curvas Cl(alpha) y Cm(alpha) viscoso vs. no viscoso")

# =====
# RESUMEN FINAL
# =====
print(f"\n{' '*70}")
print(f" RESUMEN DE RESULTADOS")
print(f"{' '*70}")
print(f" Perfil base: {nombre_perfil}")
print(f"    Cl = {Cl_base:.6f},    Cm_c/4 = {Cm_base:.6f}")
print(f" Perfil optimizado: {nombre_nuevo}")
print(f"    Cl = {Cl_new:.6f},    Cm_c/4 = {Cm_new:.6f}")
print(f"    Delta_Cl = {Cl_new - Cl_base:.4f}")
print(f"    Delta|Cm_c/4| = {abs(Cm_new) - abs(Cm_base):.6f}")
print(f" N_opt = {N_opt} paneles")
print(f"{' '*70}")

if __name__ == "__main__":
    main()

```